Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Пензенский государственный университет»
Кафедра «Вычислительная техника»

ОТЧЁТ

по лабораторной работе №7
по дисциплине «Логика и основы алгоритмизации
в инженерных задачах»
на тему: «Обход графа в глубину»

Выполнил студент гр. 25ВВВ1
Садыков Д. Р.

Проверили:
к.т.н., доцент Юрова О. В.
ст. преподаватель Деев М. В.

Пенза 2026

1. Цель работы

Изучение принципов поиска в глубину (Depth-First Search, DFS) и освоение методов программной реализации обхода графа. В рамках работы требуется реализовать рекурсивные алгоритмы для матричного и списочного представлений графа, а также нерекурсивную версию с использованием явного стека на языке Си.

2. Постановка задачи

- **Задание 1:** Сгенерировать случайный неориентированный граф без петель (в виде матрицы смежности), вывести его на экран и выполнить обход в глубину по алгоритму ПОГ.
- **Задание 1 (3*):** Реализовать ту же процедуру обхода для графа, представленного в виде списков смежности.
- **Задание 2*:** Переписать рекурсивный алгоритм обхода для матрицы смежности в нерекурсивный, используя собственный стек вызовов.

3. Структура программного решения

Приложение написано на языке Си. Для представления графа используются две структуры данных:

- `struct Graph` — хранит матрицу смежности в виде двумерного динамического массива `adj` и количество вершин `n`.
- `struct AdjList` и `struct Node` — массивы указателей на связные списки, реализующие списки смежности.

Алгоритм DFS реализован в трех вариантах:

- `pog_matrix_rec` — внешний цикл ПОГ и рекурсивная функция `dfs_matrix_rec` (обход по строкам матрицы);
- `pog_list_rec` — внешний цикл ПОГ и рекурсивная функция `dfs_list_rec` (обход по связным спискам);
- `pog_matrix_stack` — итеративный алгоритм с использованием массива-стека для эмуляции системного стека вызовов.

4. Результаты выполнения работы

4.1. Тестирование на эталонном графе

В качестве контрольного примера был взят граф из методических указаний, состоящий из 6 вершин и 5 ребер. Матрица смежности этого графа и порядок обхода представлены ниже.

```
Исходная матрица смежности (эталонный граф):

      v1  v2  v3  v4  v5  v6

 v1    0   1   1   0   0   1

 v2    1   0   0   1   0   0

 v3    1   0   0   0   1   0

 v4    0   1   0   0   0   0

 v5    0   0   1   0   0   0

 v6    1   0   0   0   0   0



Обход DFS (матрица, рекурсия) от v1: 1 2 4 3 5 6

Обход DFS (списки, рекурсия) от v1: 1 2 4 3 5 6

Обход DFS (матрица, стек) от v1:   1 2 4 3 5 6
```

Как видно из протокола, все три реализации демонстрируют идентичный порядок обхода. Логика алгоритма следующая: из вершины 1 мы переходим в первую доступную смежную вершину 2. Из 2 идем в 4. Из 4 возвращаемся обратно в 2 (так как смежных непосещенных вершин нет). Возвращаемся в 1, идем в 3, затем в 5, возвращаемся и, наконец, посещаем 6.

4.2. Тестирование на случайном графе

Генератор случайных чисел создал граф из 6 вершин с вероятностью появления ребра 40%. Ниже приведены результаты обхода для сгенерированного графа.

```
Сгенерированная матрица смежности:

      v1  v2  v3  v4  v5  v6

 v1    0   0   0   0   1   1

 v2    0   0   0   1   0   0

 v3    0   0   0   0   1   0

 v4    0   1   0   0   1   0

 v5    1   0   1   1   0   0

 v6    1   0   0   0   0   0



Обход DFS (матрица, рекурсия) от v1: 1 5 3 4 2 6
```

```
Обход DFS (списки, рекурсия) от v1: 1 5 3 4 2 6


Обход DFS (матрица, стек) от v1:   1 5 3 4 2 6
```

4.3. Сравнительный анализ реализаций

По результатам работы программы был проведен анализ эффективности различных методов представления графа.

Метод	Структура данных	Временная сложность	Особенности
Рекурсивный (матрица)	Двумерный массив	$O(V^2)$	Прост в реализации, но при больших V происходит полный перебор всех ячеек строки.
Рекурсивный (списки)	Связные списки	$O(V + E)$	Наиболее эффективен для разреженных графов. Время тратится только на реальные ребра.
Нерекурсивный (стек)	Массив + стек	$O(V^2)$	Защищает от ошибки Stack Overflow при глубокой рекурсии. Полностью имитирует системный стек.

4.4. Проверка связности (внешний цикл ПОГ)

Алгоритм ПОГ (Поиск в глубину) подразумевает обход всех вершин графа, даже если он несвязный. В программе это реализовано через внешний цикл: если после вызова `dfs()` остались вершины с флагом `num[i] == 0`, алгоритм запускается снова для них. Это гарантирует, что будут посещены все компоненты связности.

5. Выводы

- В ходе работы был изучен и успешно запрограммирован алгоритм поиска в глубину (DFS), позволяющий обходить вершины графа с возвратом назад при достижении тупика.

- Экспериментально доказано, что рекурсивный обход по матрице смежности и по спискам смежности выдает одинаковый порядок вершин при условии сортировки соседей.
- Выявлено преимущество списков смежности перед матрицей: сложность обхода снижается с $O(V^2)$ до $O(V + E)$, что критически важно для графов с большим числом вершин и малым числом ребер.
- Реализация нерекурсивного алгоритма с явным стеком подтвердила возможность исключения рисков переполнения системного стека (stack overflow) без изменения логики обхода.
- Все три реализованные функции корректно обрабатывают несвязные графы благодаря внешнему циклу проверки непосещенных вершин.